

TaxLab Salary Tax Calculator

Overview

TaxLab will provide a simple salary tax calculator that allows any user to enter an annual salary and immediately see:

- total tax owed
- take-home pay

Goal

Build a tool in TaxLab that lets users enter their salary and instantly see how much tax they owe.

Scope

In scope

- Create a worker that imports tax rate data from a spreadsheet into a database
- Create a `POST /tax-calculations` endpoint that:
 - accepts annual salary as input
 - calculates total tax across configured tax bands
 - returns total tax and take-home pay

Out of scope

- Authorization for the endpoint
- User login or authentication flows
- UI implementation in the first milestone

Proposed Solution

Tax rate ingestion

A worker will be created to read tax rate data from a spreadsheet and insert it into a database table.

The system will not read directly from the spreadsheet at request time.

Why import spreadsheet data into a database

Benefits

- Database lookups are faster and more reliable than reading from a spreadsheet on every request
- The solution avoids runtime dependency on an external spreadsheet service
- The design is easier to scale as traffic grows
- Future enhancements such as versioning and historical tax rates can be added more easily

Drawbacks of using the spreadsheet directly at runtime

- Additional authorization complexity
- Risk of rate limiting
- Spreadsheet load on every request
- Runtime dependency on external network access
- A spreadsheet cell change could break functionality immediately

Data Model

Tax rate calculation data will be stored in the database.

Table: **TaxRateTable**

Column	Type	Notes
TaxRateId	Guid	Primary key
BandStart	decimal	Inclusive lower bound for the tax band
BandFinish	decimal?	Inclusive or open-ended upper bound; null means no upper limit
TaxRate	decimal	Tax rate for the band
CreatedDateUTC	datetime	Record creation timestamp

Notes

- `BandFinish = null` represents the top tax band with no upper limit.
- Tax bands should be seeded or imported in a deterministic order to ensure predictable calculations.

API Design

A new endpoint will be created to calculate tax from an annual salary.

Endpoint

`POST /tax-calculations`

Request

```
{  
  "annualSalary": 0  
}
```

Request contract

Field	Type	Required	Notes
<code>annualSalary</code>	<code>decimal</code>	Yes	Annual salary to calculate tax for

Response

```
{  
  "totalTax": 0,  
  "takeHomePay": 0  
}
```

Response contract

Field	Type	Notes
<code>totalTax</code>	<code>decimal</code>	Total tax calculated across all applicable bands
<code>takeHomePay</code>	<code>decimal</code>	<code>annualSalary</code> - <code>totalTax</code>

Calculation Rules

Base rules

- If `annualSalary < 0`:
 - `totalTax = 0`
 - `takeHomePay = annualSalary`
- Otherwise, calculate tax across all applicable bands where:
 - `BandStart <= annualSalary`

Per-band calculation

For each applicable band, taxable amount is:

`min(annualSalary, BandFinish ?? annualSalary) - BandStart`

Tax for each band is:

`taxableAmount * TaxRate`

Total tax is the sum of tax across all applicable bands.

Take-home pay is:

`annualSalary - totalTax`

Assumptions

- `BandStart` is inclusive.
- `BandFinish` is nullable for the final band.
- Tax bands are expected to be non-overlapping and contiguous.

Rounding and Precision

Rounding behaviour is currently not fully defined and requires clarification.

Current observations

- The intended calculation appears to use 2 decimal precision while calculating each bracket.
- There is a discrepancy in one example test case:
 - salary: `125,368.25`
 - expected tax in the example: `33,335.00`
 - mathematically derived amount noted in the draft: `35,335.73`

Open question

It is currently unclear whether the API should:

- return full decimal values
- round to a fixed number of decimal places
- truncate decimal values
- round to whole dollars only

Until clarified, I will round calculate each band with full precision and then round total as to 2dp

Non-Functional Considerations

Scalability

For the current scope, a database-backed design is sufficient.

Future improvements could include:

- moving the database to AWS RDS
- adding read replicas for scale
- introducing caching for frequently requested tax calculations
- placing the API behind an API Gateway

Future Scope

Possible future enhancements include:

- allowing users to calculate tax by annual, monthly, fortnightly, or hourly income
- supporting additional tax-related scenarios such as:
 - KiwiSaver
 - secondary income
 - parental income
- supporting versioned tax tables
- delivering the UI experience inside TaxLab

Delivery Plan

Milestone 1

- Build the worker to import tax rates from the spreadsheet into the database
- Build the `POST /tax-calculations` endpoint
- Return correct `totalTax` and annual take-home pay

Milestone 2

- Design and implement the UI in TaxLab

Testing Strategy

Automated testing

Unit tests will be added to cover:

- negative salary
- zero salary
- salary within a single band

- salary spanning multiple bands
- salary at band boundaries
- top-band calculations where `BandFinish` is `null`
- rounding and precision behaviour once product rules are confirmed

Manual testing

- Manual API verification will be performed using Bruno.

Technology Choices

Backend

- C#
- EF Core

Frontend

The UI framework is not yet decided. Candidate options are:

- Angular
- React
- Vue

Open Questions

- What is the exact rounding or truncation rule for `totalTax` and `takeHomePay`?